

Table of Contents

Table of Contents	1
Course Overview	2
Computers & Programs	4
1.0 First Program	13
1.1 Output.....	24
1.2 Input & Variables	35
1.3 Expressions.....	46
1.4 Functions.....	58
1.5 Boolean Expressions	76
1.6 IF Statements.....	86
Unit 1 Practice Test	99
Unit 1 Project : Monthly Budget	101
2.0 Modularization	108
2.1 Debugging.....	130
2.2 Designing Algorithms	145
2.3 Loop Syntax	156
2.4 Loop Output	168
2.5 Loop Design.....	181
2.6 Files.....	190
Unit 2 Practice Test	207
Unit 2 Project : Calendar Program.....	209
3.0 Array Syntax	214
3.1 Array Design	233
3.2 Strings	242
3.3 Pointers	255
3.4 Pointer Arithmetic	269
3.5 Advanced Conditionals	284
Unit 3 Practice Test	301
Unit 3 Project : MadLib	303
4.0 Multi-Dimensional Arrays.....	309
4.1 Allocating Memory	324
4.2 String Class.....	337
4.3 Command Line.....	346
4.4 Instrumentation	356
Unit 4 Practice Test	361
Unit 4 Project : Sudoku	363
A. Elements of Style.....	369
B. Order of Operations	375
C. Lab Help.....	376
D. Emacs & Linux Cheat-Sheet	377
E. C++ Syntax Reference Guide	378
F. Glossary.....	381
G. Index	393

Course Overview

Procedural programming is the process of solving programming challenges by breaking large problems into smaller ones. These sub-problems are called procedures.

Goals

The goal of this class is that each student will be able to solve problems in C++ and have a solid foundation in software development methodology. By the end of the semester...

- You will be well prepared for CS 165 and the other course in computing majors. This class is the foundation on which all Computer Science (CS), Electrical Engineering (EE), and Electrical & Computer Engineering (ECEN) courses are based.
- You will have confidence in your ability to solve problems with C++. This class will include tons of opportunities for hands-on programming. By the end of the semester, you will have written more than thirty programs.
- You will possess a tool-bag of different approaches to solve software problems. Not only will we learn the mechanics of the C++ programming language, we will also learn how to solve programming problems using procedural tools.
- You will have a firm foundation in the basic constructs of procedural C++. All the components of procedural programming (except structures) will be learned in CS 124. Object Oriented Programming, the second half of the C++ language, is the topic of next semester.

These goals will be explored in the context of C++ using the Linux operating system.

Course Layout

The course will be broken into four sections:

1. **Simple Programs.** How to write your first program and begin thinking like a programmer. At the end of this section, you will be able to write a program with multiple functions, IF statements, and complex mathematical operations.
2. **Loops.** The goal of this section is to be able to solve problems using loops. Additionally we will learn several tools enabling us to solve hard programming problems and manage programs with large amounts of complexity.
3. **Arrays.** Next we will learn how to handle and manipulate large amounts of data in increasingly complex data structures. Specifically, we will learn about arrays, pointers, and file I/O.
4. **Advanced Topics.** The final section of the course will involve learning about a collection of specialized topics (dynamic memory allocation, multi-dimensional arrays, jagged arrays, and instrumentation to name a few) in the hopes they will help us understand the main topics of the semester better. In other words, the goal is not to learn these specialized topics so much as to make sure we have nailed the fundamentals.

How to Use This Textbook

This textbook is closely aligned with CS 124. All the topics, problems, and technology used in CS 124 are described in detail with these pages.

Sam and Sue

You may notice two characters present in various sections of this text. They embody two common archetypes representative of people you will probably encounter in industry.



Sue's Tips

Sue is a pragmatist. She cares only about getting the job done at a high quality level and with the minimum amount of effort. In other words, she cares little for the art of programming, focusing instead on the engineering of the craft.

Sue's Tips tend to focus on the following topics:

- Pitfalls: How to avoid common programming pitfalls that cause bugs
- Effort: How to do the same work with less time and effort
- Robustness: How to make code more resistant to bugs
- Efficiency: How to make code execute with fewer resources

Sam's Corner



Sam is a technology nerd. He likes solving hard problems for their own sake, not necessarily because they even need to be solved. Sam enjoys getting to the heart of a problem and finding the most elegant solution. It is easy for Sam to get hung up on a problem for hours even though he found a working solution long ago.

The following topics are commonly discussed in Sam's Corner:

- Details: The details of how various operations work, even though this knowledge is not necessary to get the job done
- Tidbits: Interesting tidbits explaining why things are the way they are

Neither Sue's Tips nor Sam's Corner are required knowledge for this course. However, you may discover your inner Sam or Sue and find yourself reading one of their columns.

Needing Help

Occasionally, each of us reaches a roadblock or feels like help is needed. This textbook offers two ways to bail yourself out of trouble.

If you find you are not able to understand the problems we do in class, work through them at home before class. This will give you time for reflection and help ask better questions in class.

If you find the programming problems to be too hard, take the time to type out all the examples by hand. Once you finish them, work through the challenge associated with each example. There is something about typing code by hand that helps it seep into our brains. I hope this helps.